Notes on Basic Hyperparameter Search Algorithms

Luis Mendez

Overview

Once we have a way to evaluate a hyperparameter combination reliably (cross-validation, see previous notes), we need a strategy to decide *which* combinations to try. The three basic strategies, in increasing order of automation, are manual search, grid search, and random search. All of them treat the model as a black box: pick a combination, run cross-validation, record the score, repeat. They differ only in how the next combination to try is chosen. More advanced methods (Bayesian optimization, SMBO, multi-fidelity) build on top of these by using the results of past evaluations to choose the next point intelligently, rather than exhaustively or randomly.

Manual Search

Manual search means a human iteratively picks hyperparameter values based on intuition, documentation defaults, and inspection of the resulting performance (train/validation curves, over/underfitting behavior), adjusting one or a few hyperparameters at a time between runs.

- **Pros**: cheap when only a couple of hyperparameters matter, incorporates domain knowledge, useful for building intuition about how a hyperparameter affects a specific model/dataset (e.g., how max_depth trades off bias and variance for a random forest).
- **Cons**: does not scale beyond a handful of hyperparameters, not reproducible/systematic, easy to stop at a local optimum because we stop exploring once we find "good enough," and it does not parallelize.
- In practice, manual search is often used first, to narrow down a sensible range for each hyperparameter, before handing that range to grid or random search.

Grid Search

Grid search defines a discrete set of candidate values for each hyperparameter and evaluates every combination in the Cartesian product of those sets. If hyperparameter i has n_i candidate values, the total number of combinations evaluated is

$$N = \prod_{i=1}^{p} n_i, \tag{1}$$

and, combined with k-fold cross-validation, the total number of model fits is $N \times k$. This is what makes grid search expensive: it grows exponentially with the number of hyperparameters (the "curse of dimensionality" of the search space).

Behavior and diagnostics

Because grid search evaluates a fixed, evenly-spaced set of values, plotting CV score against a single hyperparameter (holding others at their best found value) shows where the optimum roughly lies, e.g., "the optimal value seems to be between 60 and 100," which then suggests refining the grid around that range in a second pass (coarse-to-fine grid search). A flat curve across a hyperparameter's range indicates that parameter has little effect on performance for this problem, and it can be dropped from the search to save computation.

Nested / pipeline hyperparameter spaces

When the estimator being tuned is a `Pipeline` (e.g., a preprocessing step feeding into a classifier), the hyperparameter grid must reference which step each parameter belongs to. Scikit-learn uses the `step__parameter` naming convention (double underscore) to address hyperparameters nested inside pipeline steps or inside meta-estimators, e.g., `"classifier__n_estimators"` or `"classifier__max_depth"` for a `GradientBoostingClassifier` that is a named step called `classifier` inside a pipeline. This lets grid search jointly tune preprocessing choices and model hyperparameters in a single, correctly cross-validated search, avoiding data leakage that would occur if preprocessing were fit once outside the CV loop.

Pros / Cons

- **Pros**: exhaustive over the specified grid, guaranteed to find the best combination *within that grid*, simple and reproducible, trivially parallelizable (each combination is independent).
- **Cons**: cost grows exponentially with the number of hyperparameters, wastes evaluations on unimportant hyperparameters (it spends as many trials on a parameter that does not matter as on one that does, since it always covers the full Cartesian product), and resolution is limited by the discrete grid chosen up front.

Random Search

Random search samples a fixed number of combinations, N, from a specified distribution over each hyperparameter (uniform, log-uniform, discrete, etc.) instead of evaluating every combination on a grid. The search budget N is chosen directly (e.g., "try 50 random combinations") rather than being determined by the grid size.

Why it tends to outperform grid search

Bergstra and Bengio (2012) showed that random search is more efficient than grid search when only a few hyperparameters actually affect performance (which is the common case). Grid search spends a fixed number of distinct values on every hyperparameter, including unimportant ones. Random search instead explores many more distinct values along each individual hyperparameter's axis for the same total number of trials N, because the values are not constrained to a shared grid. This means random search is more likely to land close to the optimum of the hyperparameters that actually matter, without needing to know in advance which ones those are.

sklearn implementation

`RandomizedSearchCV` takes a `param_distributions` dictionary mapping each hyperparameter to either a list of discrete values (sampled uniformly) or a distribution object (e.g., `scipy.stats.uniform`, `loguniform`) to sample continuous values from, plus `n_iter` for the number of combinations to sample and cross-validate. Nested pipeline hyperparameters use the same `step__parameter` naming convention as grid search.

Pros / Cons

- **Pros**: search budget N is decoupled from the number of hyperparameters and their cardinality, so it scales far better to high-dimensional hyperparameter spaces; can sample continuous distributions, giving finer resolution than a manually chosen grid; parallelizable like grid search; easy to extend the budget incrementally (just draw more samples).
- **Cons**: not exhaustive, so it can miss the true optimum, and results have some run-to-run variance from the random sampling (mitigated by fixing a random seed and/or increasing N); still does not use information from earlier trials to guide later ones, unlike Bayesian/SMBO methods.

Grid Search vs. Random Search vs. Manual Search

- All three are *uninformed* search strategies: the choice of the next combination to evaluate does not depend on the scores observed so far (aside from a human manually adjusting between rounds).
- Manual search does not scale but injects domain knowledge; grid and random search scale better but treat the model as a pure black box.
- Grid search is preferable when the space is low-dimensional and we care about exhaustively covering a specific, small set of candidate values (e.g., final confirmation around a known-good region).
- Random search is preferable as dimensionality grows, or when we are unsure in advance which hyperparameters matter, since it allocates the trial budget more efficiently across dimensions.
- Both are naturally embarrassingly parallel, since each combination/sample is evaluated independently, and both act as the baseline that more sophisticated methods like Bayesian optimization are compared against.